Interview questions

14 repository-specific questions with answers and follow-up prompts.

Start with a concise explanation, trace a concrete example, then name the tradeoff.

ARCHITECTURE

1. Walk us through the system.

A separate Node sensor generates readings and detects crossings. A relay validates and stores alerts and settings, then distributes updates over WebSocket. The monitor displays live data and edits settings; operators acknowledge alarms in the control room.

> **Be ready for the follow-up**
>
> Follow one reading all the way to the chart. Keep detection in the sensor and durable shared state in the relay.

ARCHITECTURE

2. Why is the sensor separate from the browser?

Detection must continue when no tab is open. One source also ensures every monitor observes the same readings and alerts. The DataSource interface allows replacing the simulation with a hardware adapter.

> **Be ready for the follow-up**
>
> What changes when you introduce multiple sensors? Consider message identity, settings, connections, and source selection.

DOMAIN

3. Explain hysteresis using actual numbers.

With a threshold of 25°C and hysteresis of 0.6°C, a below state switches only above 25.3°C. An above state switches only below 24.7°C. Values exactly on an edge do not switch. The first sample never alerts.

> **Be ready for the follow-up**
>
> What happens when settings change? The agent silently reclassifies the previous value against the new threshold before evaluating the next reading.

DELIVERY

4. What if the relay stores an alert but accepted is lost?

The sensor retries with the same UUID. The relay finds the stored record and confirms it with duplicate: true, without broadcasting a second alert. Duplicate transmission is expected; duplicate storage is prevented.

> **Be ready for the follow-up**
>
> Where can alerts still be lost? Queue overflow, sensor process exit, and terminal connection/configuration failures qualify the delivery guarantee.

DELIVERY

5. Does reconnect recover everything?

No. welcome is a bounded snapshot of 100 recent alerts. The monitor replaces its collection; the control room merges it. A missed acknowledgement outside the snapshot can leave a retained alarm stale.

> **Be ready for the follow-up**
>
> A durable sequence of alert changes would support replay. Snapshot and replay boundaries must be consistent, and clients must apply pages before advancing a cursor.

REACT

6. Why is the connection in a ref and status in state?

Changing status should update the interface. The connection object is an imperative resource used for sending and cleanup; replacing that reference should not itself trigger rendering.

> **Be ready for the follow-up**
>
> Explain why creating a socket during render is unsafe and how effect cleanup handles the development StrictMode setup/cleanup check.

REACT

7. Why use useEffectEvent in useRelay?

Connection callbacks need current handlers without reconnecting whenever the component renders. Effect Events provide access to current handlers while the connection effect depends on the stable connect function.

> **Be ready for the follow-up**
>
> A stale closure retains values from an older render. Effect Events are not a general excuse to omit legitimate effect dependencies.

REACT

8. Why use useSyncExternalStore for readings?

Readings live in an external ring buffer. useSyncExternalStore subscribes React to consistent snapshots. The chart reads the window; the readout reads the latest value. Snapshots keep identity until data changes.

> **Be ready for the follow-up**
>
> Redux could support this sampling rate too. The split deliberately isolates frequent sample updates rather than proving Redux cannot handle them.

REACT

9. Why Redux in one app and useReducer in the other?

The monitor has shared alert, setting, and sensor slices with derived selectors. The control room has one central list with explicit transitions, so a reducer is enough. Local input drafts stay in component state.

> **Be ready for the follow-up**
>
> Avoid choosing a state library just because it is familiar. Explain state scope, update frequency, and who subscribes.

STORAGE

10. Why SQLite, and what limits scaling?

It fits one relay process with simple durable storage and transactions. Conditional updates enforce first acknowledgement wins. Synchronous operations and local-file deployment need reassessment as workload grows.

> **Be ready for the follow-up**
>
> Multiple relays also need shared event distribution and sensor ownership coordination; replacing the database alone is insufficient.

SECURITY

11. Where does the operator name come from?

The authenticated server session, never the ack message. The message contains only the alert ID. Browser roles use a session cookie; the sensor authenticates with a service token in hello.

> **Be ready for the follow-up**
>
> Explain password hashing, hashed session tokens, origin validation, session expiry, and the difference between authentication and authorization.

TESTING

12. What do fake sockets fail to prove?

They make retry and timeout tests deterministic, but do not prove real socket behavior, browser integration, or deployment configuration. Real-relay integration, Playwright, and Docker checks cover other boundaries.

> **Be ready for the follow-up**
>
> Use one concrete example from a test you understand. Explain what the test asserts and what it leaves unverified.

TRADEOFFS

13. What would you improve first?

Reconnect recovery is a correctness fix. Stale-data labels prevent old temperatures looking current. A durable sensor outbox protects unaccepted alerts across restarts. History and retention depend on the product requirements.

> **Be ready for the follow-up**
>
> Describe one implementation path and one tradeoff. Clearly distinguish proposed work from implemented behavior.

TRADEOFFS

14. Is the latency independent of clock skew?

The alert log subtracts the sensor event time from the relay receipt time. It does not use the browser clock, but sensor/relay clock skew can still affect the number.

> **Be ready for the follow-up**
>
> The readout above/below label also differs from detector state: it compares directly with the threshold, while detection remembers state and uses hysteresis.
